

# Preregistration — Capability Composition and Confused-Deputy Resistance

---

**Experiment ID:** AUTH-002

**Series:** Authority Partitioning (AUTH)

**Parent experiment:** AUTH-001 (PASS — 18/18)

**Parent context:** IB-001 (FAIL), IB-002 (FAIL), IB-003 (PASS)

**Principal Investigator:** Derek Hone, Remnant Fieldworks Inc.

**Date locked:** 2026-08-16

**Framework:** Coherent Inheritance Framework (CIF) / ExecutionProof

**Governing sentence:** “We will accept the result that is true, not the result we hoped to see.”

---

## Lineage

---

AUTH-001 demonstrated that narrow capability grants can replace a flat authorized-action-types list without breaking legitimate workflows or creating accidental privilege inheritance — within 18 preregistered cases. Every grant was checked in isolation: one actor, one action, one authorization decision.

But real systems do not stay isolated. Agents delegate to other agents. Tools invoke tools. A read feeds a transform that feeds a transmit. Grants issued safely to individual actors can become unsafe when the system starts **composing** them — when one component’s legitimate authority is exercised on behalf of, or at the direction of, a component that should not hold that authority.

The classic failure mode is the **confused deputy**: an authorized component is tricked (or simply asked) into exercising its own legitimate capability for an unauthorized purpose or on behalf of an unauthorized requester. AUTH-001 did not test this. Its actors held their own grants and exercised them directly. AUTH-002 pressure-tests the most dangerous assumption behind AUTH-001: **that safe individual grants remain safe when the system starts composing them.**

## Research Question

---

Can individually valid narrow capabilities become unsafe when composed across agents, tools, or chained actions — through delegated authority, cross-agent invocation, capability reuse, scope intersection, stale grants, or confused-deputy patterns — and does the capability-grant model detect and deny these composition-level threats without breaking legitimate multi-agent workflows?

## Hypothesis

---

**H0 (null):** The narrow capability-grant model fails to prevent at least one composition-level threat (confused deputy, unauthorized delegation, stale-grant reuse, scope intersection leak, or cross-agent privilege laundering), or it can only prevent composition threats by also blocking at least one legitimate multi-agent workflow that holds the exact delegated capabilities it declared.

**H1 (experimental):** The capability-grant model denies every preregistered composition-level attack while permitting every preregistered legitimate multi-agent workflow that operates within its declared

delegated capabilities — with zero confused-deputy successes, zero unauthorized delegation successes, and zero stale-grant or scope-intersection leaks.

## Design — Composition Extensions to the Capability-Grant Model

AUTH-002 extends the AUTH-001 model with composition-aware rules. The base model is unchanged:

```
CapabilityGrant = {
  actor:          str
  action_type:    str          # canonical (normalized via WORDING_EQUIVALENCES)
  resource_class: str          # never "*"
  scope:          "single" | "batch"
  constraints:    dict          # e.g. {"named_tool": "reconciler"}
```

The five-part authorization test from AUTH-001 is unchanged. AUTH-002 adds the following composition rules that govern **how** grants are exercised in multi-agent contexts:

### Rule 1 — No Implicit Delegation

An actor's capability grant authorizes **only that actor**. If agent-A holds `transmit_external:partner-api:single` and agent-B asks agent-A to transmit on B's behalf, agent-A's grant authorizes the transmit only if **agent-A is the actor on the action**. There is no mechanism for B to "borrow" A's grant. Delegation must be explicit: agent-B must hold its own grant, or a delegation grant must exist that names both the delegator and the delegate.

### Rule 2 — Delegation Grant (explicit, narrow)

A **delegation grant** is an additional tuple that authorizes one actor to exercise a specific capability **on behalf of** another actor:

```
DelegationGrant = {
  delegator:      str          # who grants the delegation
  delegate:       str          # who receives it
  action_type:    str          # what action the delegate may perform
  resource_class: str          # on what resource class
  scope:          "single" | "batch"
  constraints:    dict          # same as capability constraints
```

A delegated action is authorized iff:

1. The **delegate** holds a matching CapabilityGrant for the action (the delegate must have its own standing authority).
2. A matching DelegationGrant exists naming the (delegator, delegate, action\_type, resource\_class, scope).
3. All constraints are satisfied on both the capability grant and the delegation grant.

**Critical rule:** A delegation grant does **not** transfer the delegator's capability to the delegate. The delegate must independently hold the capability. The delegation grant only authorizes the delegate to exercise that capability in contexts that serve the delegator's workflow. This prevents privilege laundering: an unprivileged actor cannot gain authority simply by being delegated to by a privileged actor.

### Rule 3 — Request-Origin Binding (Confused-Deputy Defense)

Every action records a `request_origin` — the actor that initiated the request chain. The authorization check verifies that the **executing actor** holds the capability, but if `request_origin` differs from the executing actor, a matching `DelegationGrant` must also exist. Without it, the action is denied even if the executing actor holds the capability — because the capability is being exercised at someone else's direction.

### Rule 4 — Grant Freshness

Each capability grant carries an `issued_at` timestamp and an optional `expires_at` timestamp. A grant that has expired is treated as non-existent. There is no grace period.

### Rule 5 — No Scope Intersection

Holding grants on two different resource classes does not create authority on the intersection or union. Each grant is evaluated independently. A chain that crosses resource classes requires a grant for each class — which AUTH-001 already tested in A14/A17. AUTH-002 adds adversarial cases where an attacker tries to exploit the boundary.

## Preregistered Cases

---

Actor names: `ops-agent`, `analytics-agent`, `audit-agent`, `external-requester`. All capabilities use the AUTH-001 model. **Ground truth is locked before execution and is not changed after seeing results.**

| Case | Category                                        | Scenario                                                                                                                                                                     | Expected |
|------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| B01  | Delegated authority — legit                     | ops-agent delegates transmit to analytics-agent; analytics-agent holds transmit grant + delegation grant exists                                                              | ALLOW    |
| B02  | Delegated authority — no delegation grant       | ops-agent asks analytics-agent to transmit; analytics-agent holds transmit grant but no delegation grant exists                                                              | DENY     |
| B03  | Delegated authority — delegate lacks capability | ops-agent delegates transmit to analytics-agent; delegation grant exists but analytics-agent does NOT hold transmit capability                                               | DENY     |
| B04  | Cross-agent invocation — legit                  | ops-agent calls audit-agent to generate audit report; audit-agent holds generate_report grant + delegation grant from ops-agent exists                                       | ALLOW    |
| B05  | Cross-agent invocation — no standing capability | ops-agent delegates delete to analytics-agent; delegation grant exists but analytics-agent has no delete capability                                                          | DENY     |
| B06  | Confused deputy — classic                       | external-requester (no grants) asks ops-agent to transmit_external on requester's behalf; ops-agent holds transmit grant but no delegation grant from external-requester ex- | DENY     |

| Case | Category                                              | Scenario                                                                                                                                                                                                         | Expected |
|------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|      |                                                       | ists. Ops-agent's capability should NOT be exercised for the unauthorized requester.                                                                                                                             |          |
| B07  | Confused deputy — tool-mediated                       | external-requester asks ops-agent to execute_tool:reconciler; ops-agent holds the tool grant but request_origin is external-requester with no delegation grant. Tool's own authority must not be laundered.      | DENY     |
| B08  | Confused deputy — chained                             | external-requester → analytics-agent → ops-agent chain; ops-agent holds the final transmit grant but request_origin traces back to external-requester who has no delegation path.                                | DENY     |
| B09  | Read→transform→transmit chain — legit                 | ops-agent reads (finance-reports), analytics-agent generates report (finance-reports) with delegation from ops-agent, ops-agent transmits (partner-api). Each step authorized with proper grants and delegation. | ALLOW    |
| B10  | Read→transform→transmit chain — transmit unauthorized | Same as B09 but ops-agent's transmit grant is missing. Chain should fail at transmit step.                                                                                                                       | DENY     |
| B11  |                                                       |                                                                                                                                                                                                                  | DENY     |

| Case | Category                                     | Scenario                                                                                                                                                                                                           | Expected |
|------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|      | Stale grant — expired capability             | ops-agent holds a delete_record grant that expired 1 hour ago; attempts to delete                                                                                                                                  |          |
| B12  | Stale grant — fresh capability               | ops-agent holds a delete_record grant that expires 1 hour from now; attempts to delete                                                                                                                             | ALLOW    |
| B13  | Stale delegation — expired delegation grant  | ops-agent delegates transmit to analytics-agent; delegation grant expired; analytics-agent holds live transmit capability                                                                                          | DENY     |
| B14  | Capability reuse — same grant, second action | ops-agent uses read_file:finance-reports:single, then attempts a second read on same class. Single-scope grant should still authorize each individual action (non-consumable).                                     | ALLOW    |
| B15  | Scope intersection attack                    | ops-agent holds write_file:ops-config:single AND read_file:finance-reports:single; attempts write_file on finance-reports. Holding write on one class + read on another does NOT create write on the second class. | DENY     |
| B16  | Privilege laundering — delegation chain      | external-requester delegates to analytics-agent, analytics-agent delegates to ops-agent. Even with                                                                                                                 | DENY     |

| Case | Category                                           | Scenario                                                                                                                                                                                                                                                                                                                                                       | Expected |
|------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|      |                                                    | a delegation chain, ops-agent's action still requires request_origin to have a valid delegation path. external-requester has no grants.                                                                                                                                                                                                                        |          |
| B17  | Valid multi-agent workflow (3 agents)              | ops-agent reads config, delegates to analytics-agent for report generation, analytics-agent delegates to audit-agent for audit logging. Each agent holds its own capability + delegation grants exist for each hop.                                                                                                                                            | ALLOW    |
| B18  | Delegation does not transfer — delegate acts alone | analytics-agent received delegation from ops-agent for transmit. analytics-agent then tries to transmit on its OWN behalf (not for ops-agent) in a separate context with no delegation. Should succeed only if analytics-agent holds its own transmit capability (which it does). Self-originated action by an actor who holds the capability is always ALLOW. | ALLOW    |

**These labels are locked. We do not change them after seeing results.**

## Capability and Delegation Grants per Case (Preregistered)

---

Detailed grant assignments are specified in the implementation. The following invariants hold across all cases:

- No actor receives a capability grant they are not declared to need.
- No delegation grant exists unless the case explicitly states it.
- No wildcard resource class (“\*”) is used anywhere.
- All timestamps are deterministic (fixed reference time).
- `external-requester` holds **zero** capability grants in every case.

## Success Condition

---

All of the following must hold simultaneously:

1. **Legitimate delegated/multi-agent workflows (B01, B04, B09, B12, B14, B17, B18):** ALLOW — 7/7.
2. **Composition-level attacks (B02, B03, B05, B06, B07, B08, B10, B11, B13, B15, B16):** DENY — 11/11.
3. **No confused-deputy case succeeds** (B06, B07, B08 all DENY).
4. **No stale-grant case succeeds** (B11, B13 both DENY).
5. **No scope-intersection or privilege-laundering case succeeds** (B15, B16 both DENY).
6. **No grant was broadened beyond its declared minimal set** to achieve any ALLOW.
7. **No capability normalization expanded privilege** (same K4 audit as AUTH-001).

**Overall: 18/18 correct, zero composition-level leaks, zero grant broadening.**

## Kill Conditions

---

The hypothesis is falsified if **any** of these occur:

1. **K1 — Composition defense breaks legitimate multi-agent work.** Any legitimate workflow (B01, B04, B09, B12, B14, B17, B18) is denied while holding exactly the capabilities and delegation grants it declared. This is the AUTH-001 K1 analog at the composition level.
2. **K2 — Confused-deputy success.** Any confused-deputy case (B06, B07, B08) returns ALLOW. This is the load-bearing kill condition — the entire point of the experiment.
3. **K3 — Delegation leak.** Any case where delegation is missing, expired, or the delegate lacks the capability (B02, B03, B05, B10, B11, B13, B16) returns ALLOW.
4. **K4 — Scope-intersection or privilege-laundering leak.** B15 or B16 returns ALLOW.
5. **K5 — Normalization expands privilege.** Same audit standard as AUTH-001/IB-series.
6. **K6 — “Just in case” breadth required.** Any ALLOW required a grant or delegation not declared in the preregistered case specification.

## Method

---

1. Extend the AUTH-001 capability-grant model with DelegationGrant, request-origin binding, and grant freshness (Rules 1–5 above).

2. Reuse the IB-003/AUTH-001 `WORDING_EQUIVALENCES` map unchanged; introduce no new synonym behavior.
3. Construct the 18 preregistered cases with their locked grant sets, delegation grants, timestamps, and ground-truth labels.
4. Evaluate each case through the extended authorization test; record the full per-case audit trail.
5. For chain/multi-agent cases, evaluate each step and record which step (if any) caused denial.
6. Run a privilege-expansion audit across all cases (K5).
7. Run a “just in case” audit: confirm every ALLOW used only declared grants/delegations (K6).
8. Evaluate all six kill conditions.
9. Determine verdict (PASS/FAIL) and preserve it regardless of outcome.

## Outputs (produced only after implementation is authorized)

---

- `auth002_results.json` — full case-by-case results with audit trail
- `proofrecord_auth002.json` — self-verifying ProofRecord with hash chain
- `auth002_ledger.jsonl` — one line per case for programmatic consumption
- `MANIFEST.sha256` — SHA-256 hashes of all files

## Scope Boundary

---

Explicitly **frozen out of AUTH-002**:

- Any change to AUTH-001 code, results, labels, or verdicts.
- Any change to IB-001/002/003 code, results, labels, or verdicts.
- Any new synonym/fuzzy matching beyond the declared `WORDING_EQUIVALENCES` map.
- Any wildcard or catch-all resource class.
- Any change to kill conditions or ground-truth labels after seeing results.
- Runtime revocation of grants (a later branch may test dynamic grant management).
- Rate limiting, quota enforcement, or time-windowed capabilities beyond simple expiration.
- Trust hierarchies or role inheritance (AUTH-002 tests flat delegation, not hierarchical authority).

## Relationship to AUTH-001

---

AUTH-001 tested: “Can the flat grant be replaced with narrow capabilities without breaking legitimate work?”

AUTH-002 tests: “Do those narrow capabilities remain safe when composed across agents?”

The questions are sequential and dependent. If AUTH-001 had failed, AUTH-002 would have no foundation. AUTH-001’s PASS is a necessary but not sufficient condition for composition safety — AUTH-002 tests the sufficient condition.

## Scientific Discipline

---

- Preregistration locked before execution; SHA-256 hash of this document recorded before any implementation is written or run.
- All verdicts preserved regardless of outcome.

- The synonym map is not modified after seeing results.
- IB-001 remains FAIL. IB-002 remains FAIL. IB-003 remains PASS. AUTH-001 remains PASS. AUTH-002 is an independent experiment and does not alter any prior result.
- This preregistration is written and locked **before** the implementation is touched.
- Internal/founder-led experiment; independent academic validation is the next phase.
- Any result is bounded to these 18 preregistered cases and does not, by itself, establish general composition safety.